

Aplicaciones web orientada a servicios

Proyecto: Plastic Recycler



Desarrollo de software en multiplataforma

Universidad Tecnológica Fidel Velázquez

Integrantes

Jovanny Aldahir Martínez Escudero

Jonathan Yael Jiménez Flores

Maria Fernanda Cañongo Martínez

Carlos Saúl Moreno Delgado

Estrella Lizeth Blancas Vázquez



**"2023. Año del Septuagésimo Aniversario del
Reconocimiento del Derecho al Voto de las Mujeres en México"**

Tipo de sitio

Sitio web tradicional con React en partes específicas

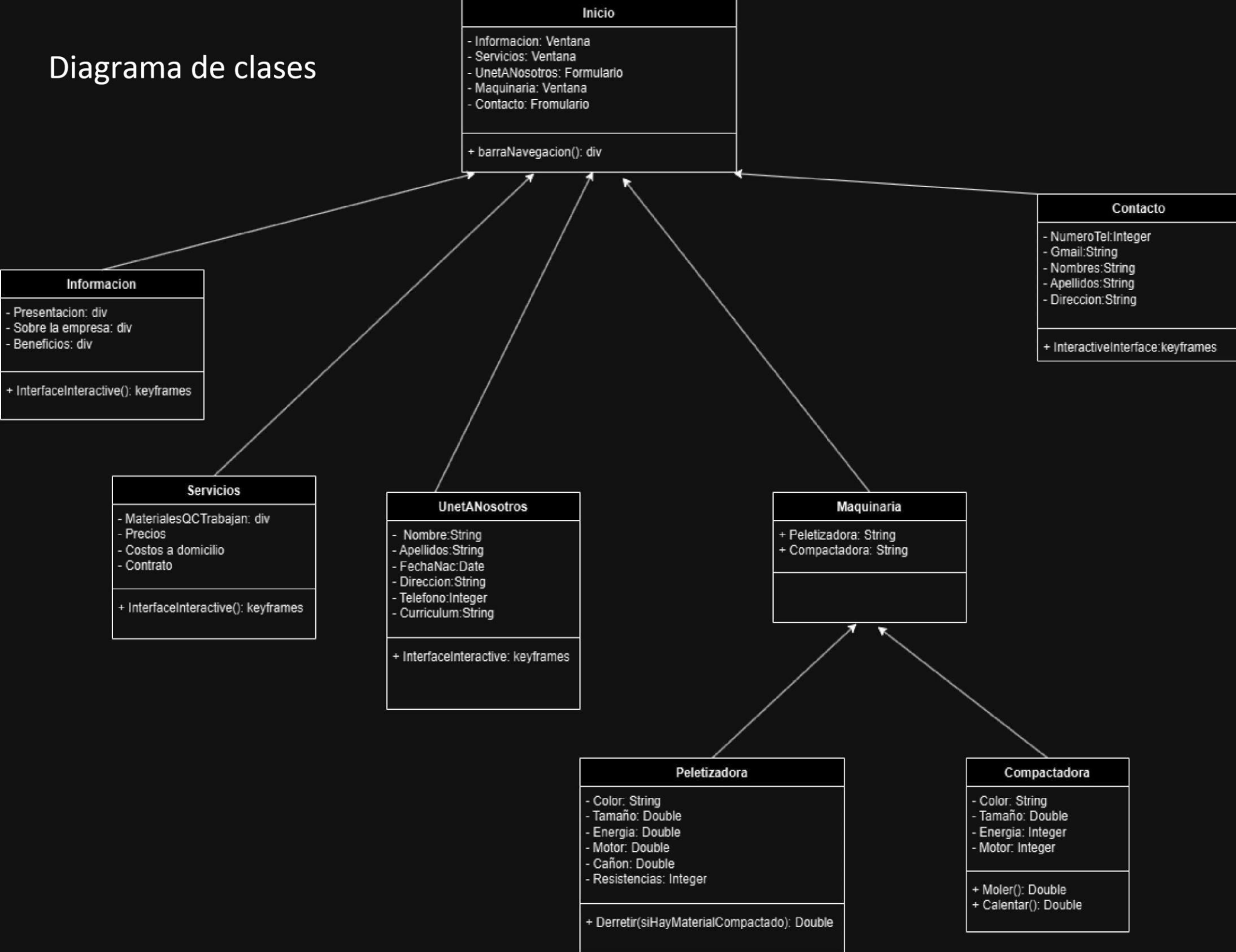
- Cada página es un archivo .html separado.
- Se usa JavaScript y React en algunas secciones para mejorar la interactividad.
- Puede incluir <script> en cada HTML para cargar React de forma independiente.

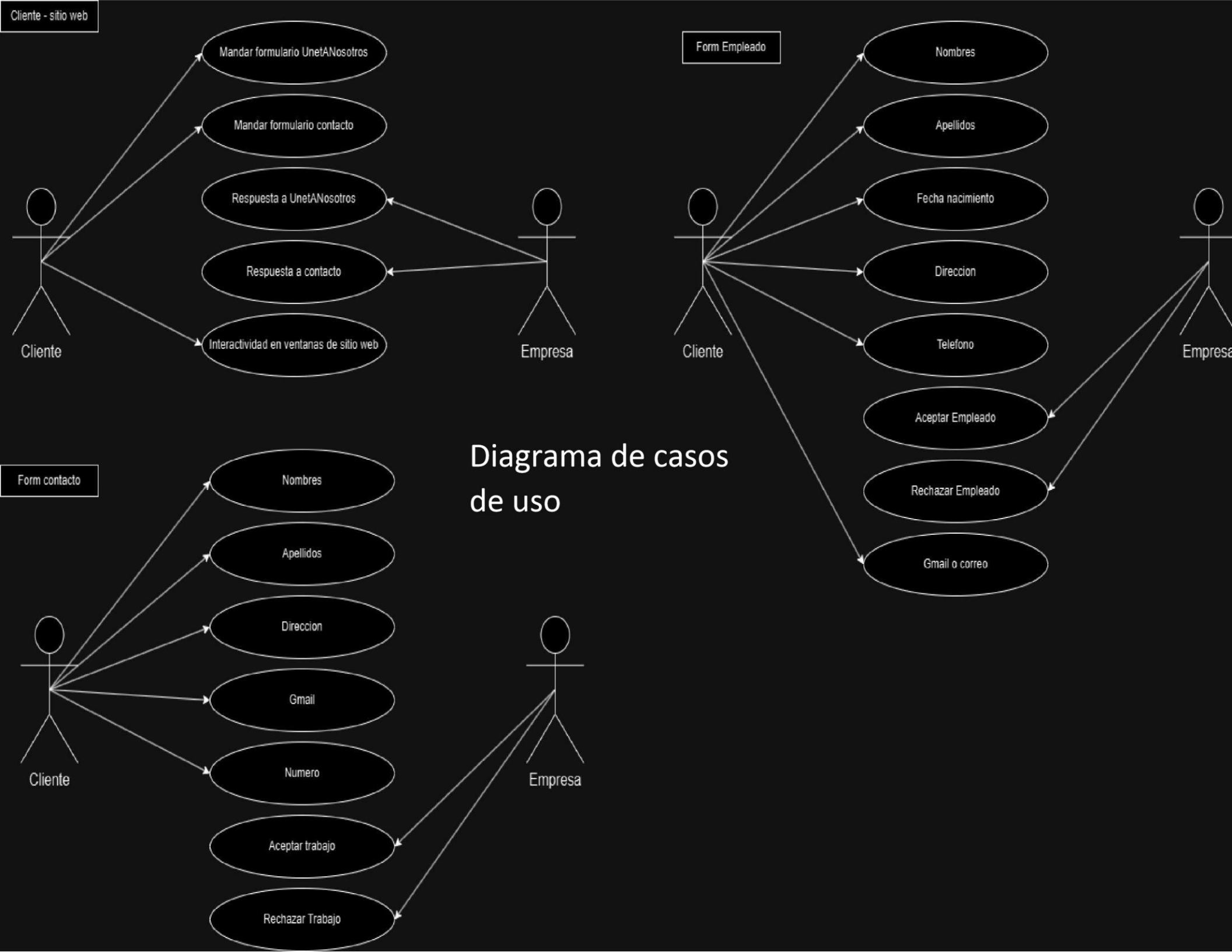
Desarrollo del Back-End con Node.js, Express y MySQL

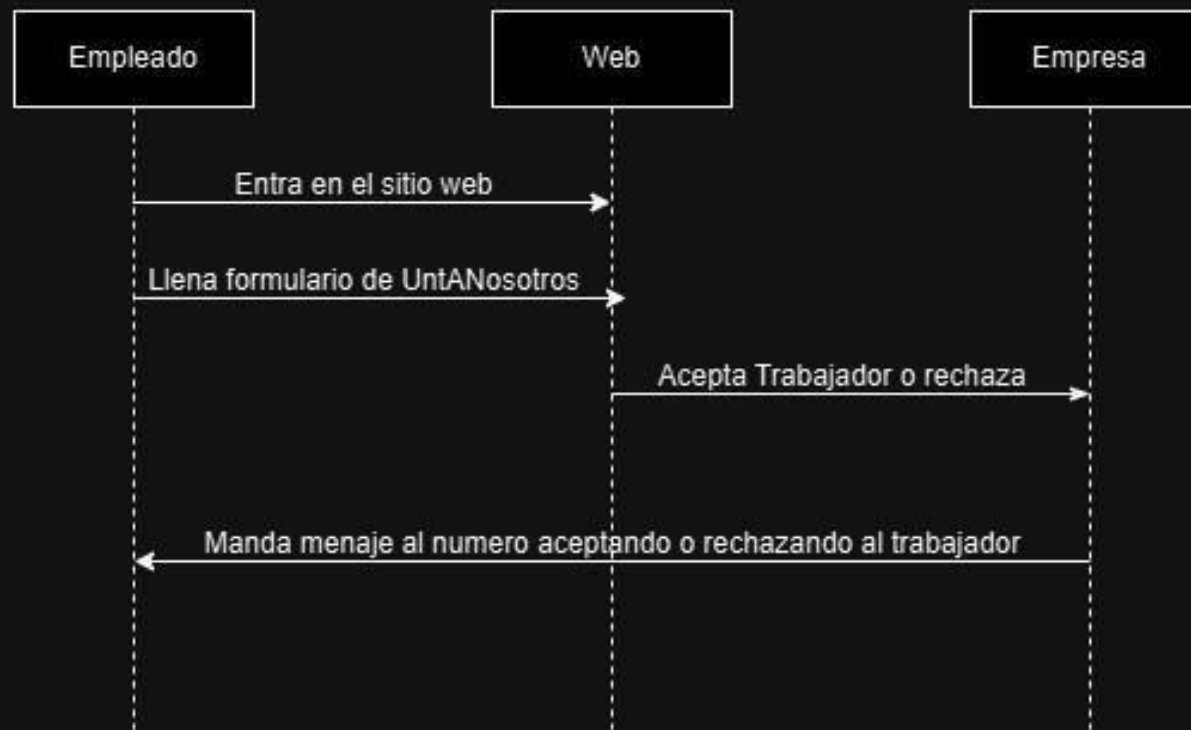
El Back-End se desarrollará utilizando Node.js con el framework Express.js para manejar las solicitudes HTTP y la comunicación con una base de datos MySQL. Se implementará una API REST, permitiendo que el Front-End (por ejemplo, una aplicación en React) interactúe con la base de datos de manera eficiente.



Diagrama de clases







**"2023. Año del Septuagésimo Aniversario del
Reconocimiento del Derecho al Voto de las Mujeres en México"**

Mapa de navegación



"2023. Año del Septuagésimo Aniversario del
Reconocimiento del Derecho al Voto de las Mujeres en México"

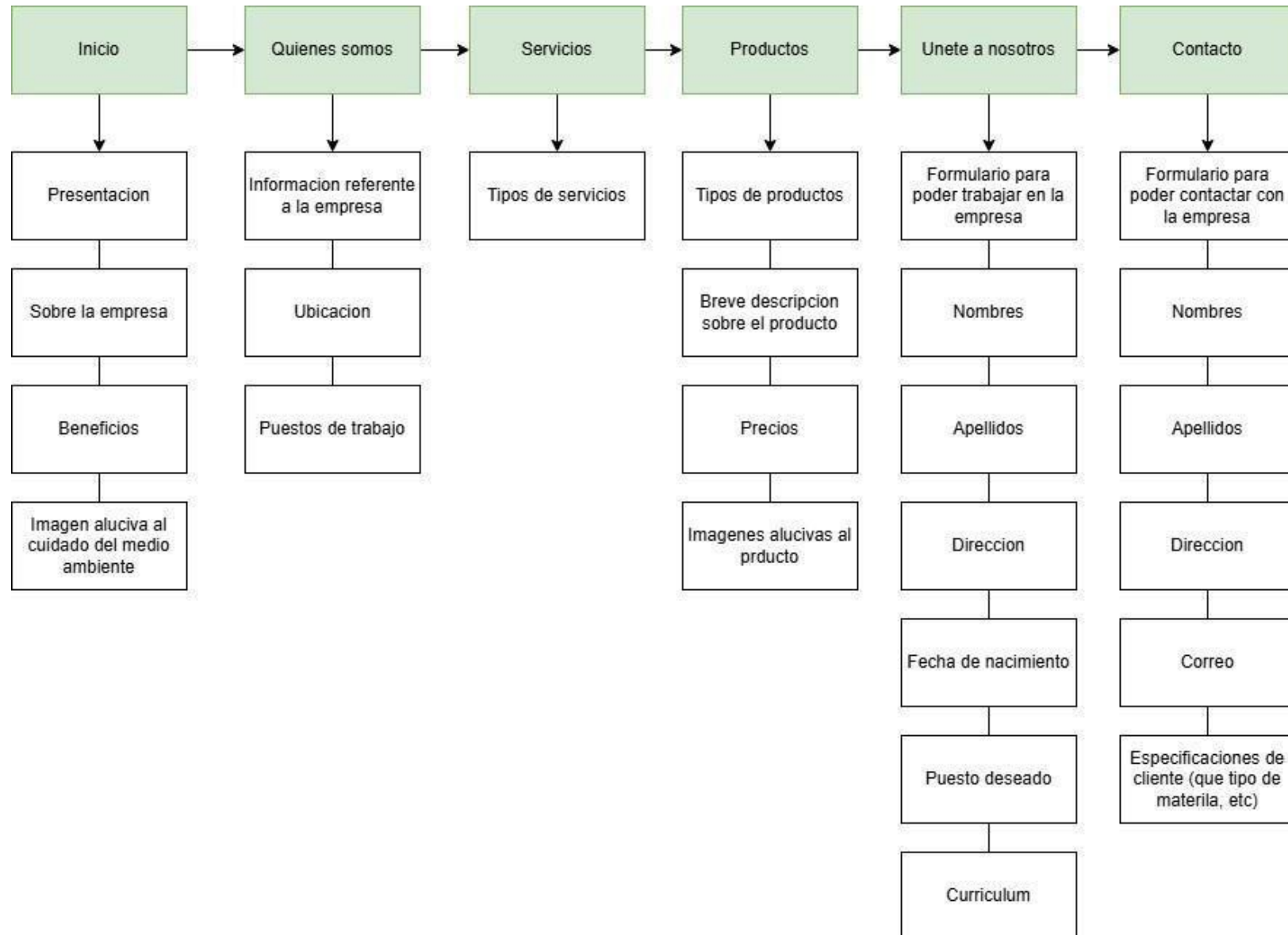
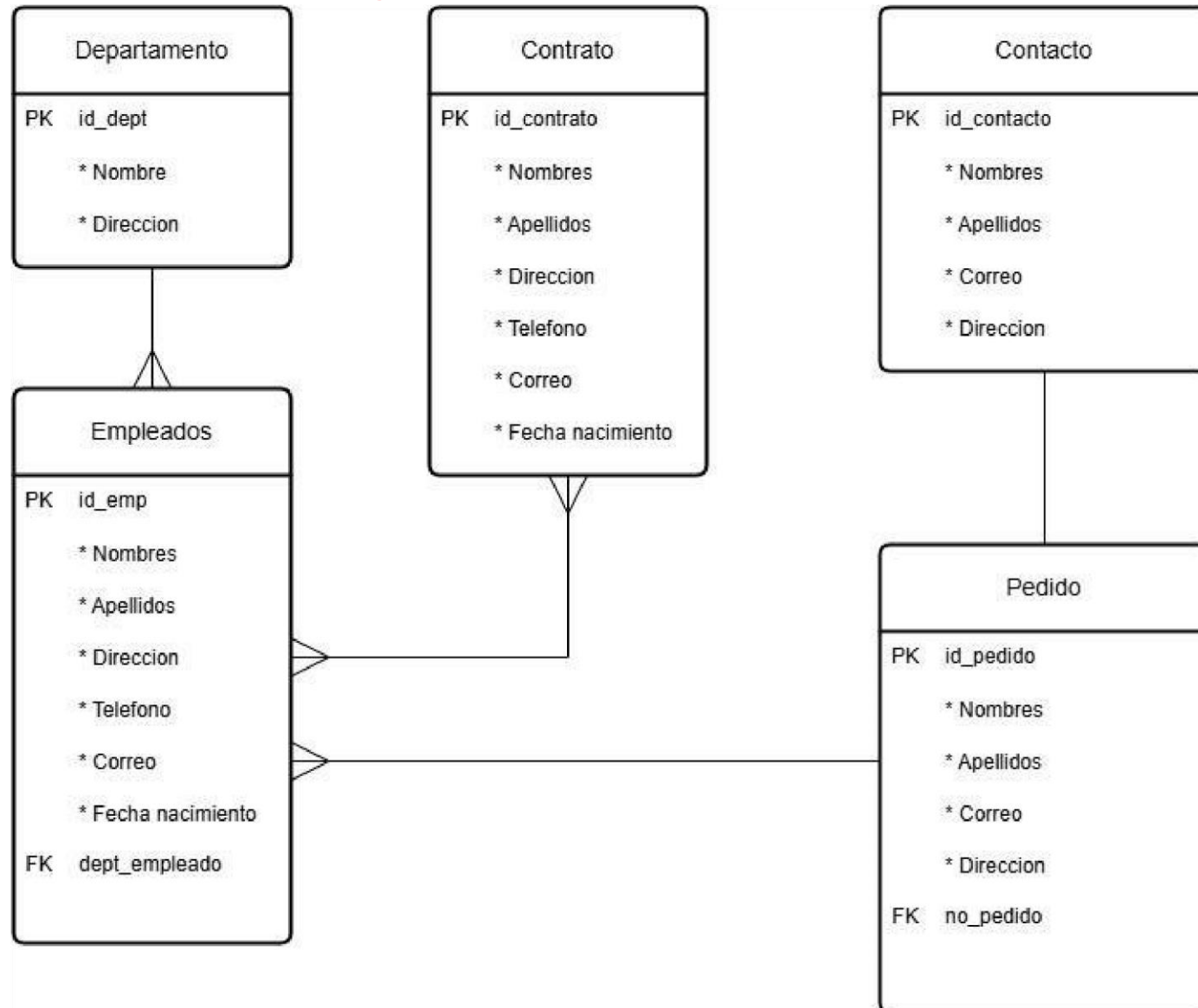


Diagrama entidad relación



Diseño GUI

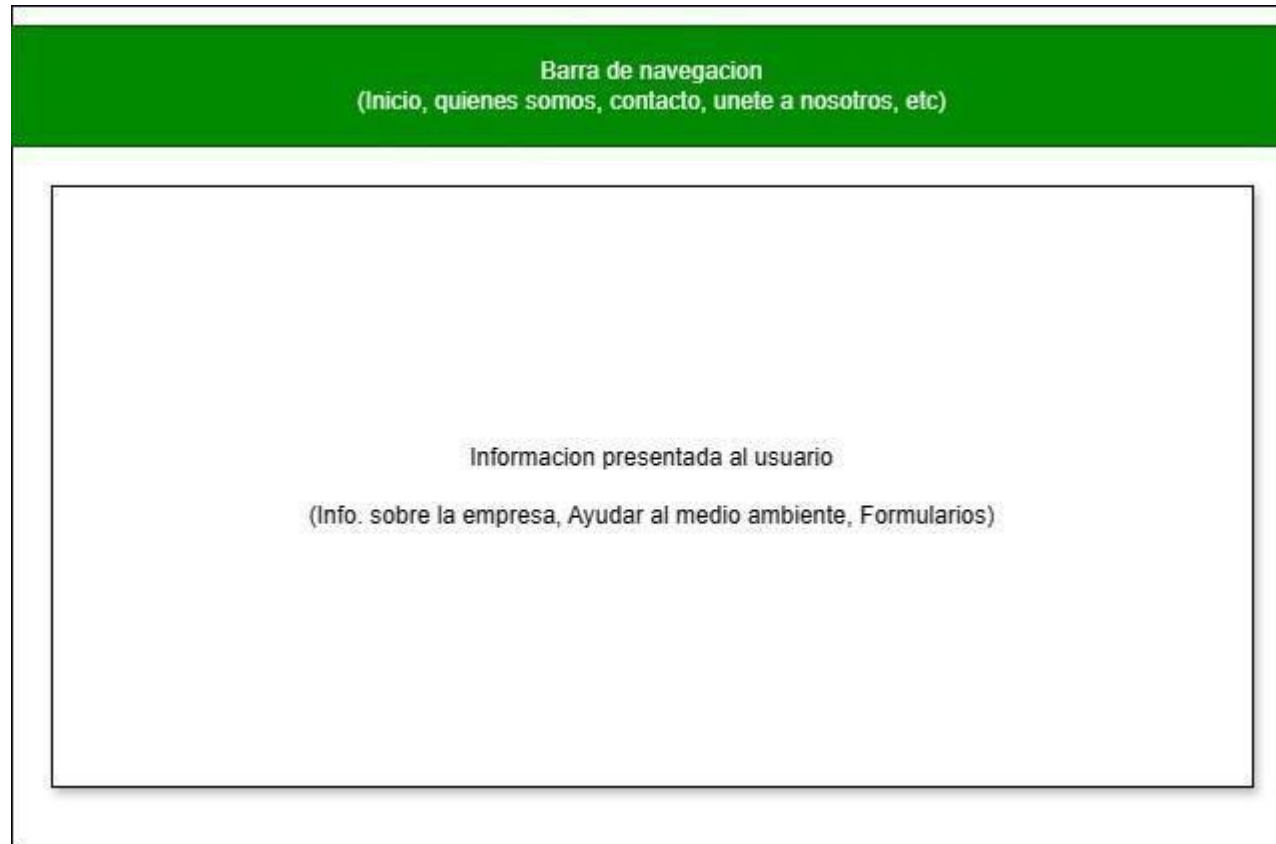


Diagrama de componentes

"2023. Año del Septuagésimo Aniversario del
Reconocimiento del Derecho al Voto de las Mujeres en México"



Frontend: Interfaz de usuario (páginas web y formularios).

Backend: Lógica de servidor (procesamiento de datos, validaciones).

Base de datos: Almacenamiento de datos de usuarios, testimonios, y contactos.

Tecnologías Utilizadas (Back-End)

Node.js: Entorno de ejecución para JavaScript en el servidor.

Express.js: Framework para crear el servidor y manejar rutas.

MySQL: Base de datos relacional para almacenar la información.

Sequelize o mysql2: Librerías para conectar Node.js con MySQL.

2. Arquitectura del Proyecto

El Back-End seguirá la arquitectura MVC (Modelo-Vista-Controlador) para mantener un código estructurado y escalable.

Rutas: Definen los endpoints de la API.

Controladores: Manejan la lógica de las peticiones.

Modelos: Representan las tablas de la base de datos en código.

Base de datos: MySQL almacena los datos de la aplicación.

3. Funcionamiento de la API REST

La API REST permitirá que el Front-End realice operaciones CRUD (Crear, Leer, Actualizar y Eliminar) sobre la base de datos.

Ejemplo de Endpoints:

GET /users → Obtener todos los usuarios.

GET /users/:id → Obtener un usuario por ID.

POST /users → Crear un nuevo usuario.

PUT /users/:id → Actualizar datos de un usuario.

DELETE /users/:id → Eliminar un usuario.

5. Despliegue y Hosting

El servidor se podrá desplegar en plataformas como Railway, Render o un VPS, asegurando su disponibilidad y acceso desde el Front-End.

Desarrollo del Front-End con JavaScript, CSS, HTML y React

El Front-End de la aplicación se desarrollará utilizando tecnologías modernas para crear una interfaz de usuario interactiva, responsiva y eficiente.

1. Tecnologías Utilizadas (Front-End)

HTML: Estructura principal de la aplicación.

CSS: Estilos y diseño responsivo para mejorar la apariencia.

JavaScript: Lógica de la aplicación y manipulación del DOM.

React: Biblioteca de JavaScript para construir interfaces dinámicas y componentes reutilizables.

2. Arquitectura del Proyecto componentes de React.

Componentes: Bloques reutilizables de la interfaz.

Estado y Props: Manejo de datos internos y comunicación entre componentes.

Rutas con React Router: Para manejar la navegación entre páginas sin recargar la
El código estará organizado de manera modular, utilizando la estructura de web.

Consumo de API: Con fetch o axios para conectar con el Back-End en Node.js.

3. Funcionamiento y Características

Interfaz intuitiva: Diseñada con HTML y CSS, asegurando una buena experiencia de usuario.

Estilos modernos: Uso de Flexbox, Grid y frameworks como Tailwind o Bootstrap si es necesario.

Dinamismo: Uso de eventos, hooks (useState, useEffect) y estado global si la aplicación lo requiere.

4. Conexión con el Back-End

El Front-End se conectará a la API REST creada en Node.js y Express para enviar y recibir datos desde la base de datos MySQL.

Ejemplo de solicitud con fetch:

javascript Copiar Editar

```
fetch('https://api.example.com/users')  
.then(response => response.json())  
.then(data => console.log(data))  
.catch(error => console.error('Error:', error));
```

5. Despliegue y Hosting

El Front-End podrá alojarse en InfinityFree, Netlify, Vercel o GitHub Pages, dependiendo de los requerimientos del proyecto.

¿Se retomara algún proyecto?

Si, se retomara el proyecto: Plastic Recycler, y se agrgara una nueva función que es un inicio de sesión además de actualizar las tecnologías con las que estaba hechas anteriormente (html, javascript. Css y php)

2. API de Productos o Servicios (Si la app maneja elementos como productos, artículos, etc.)

Funcionalidad: Gestión de productos, precios y descripciones.

Endpoints Ejemplo:

GET /products → Obtener todos los productos.

POST /products → Agregar un nuevo producto.

PUT /products/:id → Editar un producto.

DELETE /products/:id → Eliminar un producto.

Proceso en GitHub

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere?
[Import a repository.](#)

Required fields are marked with an asterisk (*).

Repository owner and name **Jovas21-P** /

Great repository names are short and memorable. Need inspiration? How about **reimagined-octo-fortnight** ?

Description (optional)

- ☒  **Public**
Anyone on the internet can see this repository. You choose who can commit.
- ☐  **Private**
You choose who can see and commit to this repository.

Initialize this repository with:

- ☐ Add a README file
This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore

.gitignore template: None ▾

Choose which files not to track from a list of templates. [Learn more about ignoring files](#)



Comandos de consola

- git init ➤ git add .
- git commit "descripción de commit"
- git push -u origin master

Conclusion

Trabajar con Node.js y React juntos es una combinación poderosa para desarrollar aplicaciones web modernas y escalables. Node.js, al ser un entorno de ejecución de JavaScript en el servidor, permite que el mismo lenguaje se utilice tanto en el backend como en el frontend, lo que mejora la eficiencia y facilita la colaboración entre los desarrolladores de ambas partes. Además, Node.js es altamente eficiente y bien adaptado para manejar aplicaciones en tiempo real y tareas asíncronas, gracias a su modelo de ejecución basado en eventos.